

Do **not** keep baseline, scaffold, and experimental implementations inside the final product tree.

Build one clean repository containing only the **final production architecture**. Preserve history through Git commits/tags outside the active source tree, then remove every duplicate, placeholder, and obsolete implementation from the release branch.

The target should be:

One hardware revision, one firmware implementation, one desktop application, one database schema, one protocol, and one documented build procedure.

Your current project has strong architecture, but several parts described as future work are still templates or stubs rather than integrated product features.

Final product target

I would define the finished system as:

OSBAMS Production System

Battery test fixture

- Protected 60 V-class DC power path
- Independent emergency shutdown
- INA228 voltage/current/power measurement
- Multiple temperature channels
- STM32 controller
- Programmable electronic-load control
- Local display and controls
- USB communication
- Local data storage
- Calibrated enclosure-mounted hardware

Desktop application

- Battery registry
- Test configuration
- Live monitoring
- Automated test control
- Database
- Engineering scoring
- ML-assisted analysis
- Reports

└─ Calibration and diagnostics

1. Delete or remove from the final source tree

After transferring anything useful into the final implementation, remove these kinds of files:

```
*_v0_1.c
*_v0_1.h
*_placeholder.c
*_scaffold.c
*_template.c
*_with_registers.c
merged_reference/
baseline/
experimental/
Docs/ copies of source code
duplicate numbered filenames
```

Examples from your current suite:

```
i2c_v0_1.c
uart_v0_1.c
tc74_v0_1.c
main_scaffold_placeholder.c
i2c_bus_with_registers.c
freertos_tasks template
```

Do not simply delete the repository history. Before cleaning:

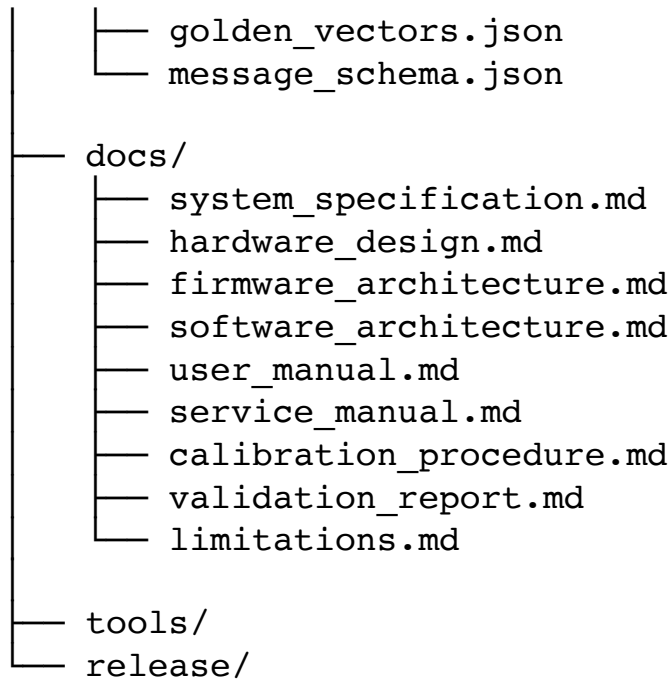
```
git tag firmware-v0.1-baseline
git tag firmware-v0.2-scaffold
```

Then the final branch contains only the production version.

2. Use one clean repository structure

I recommend:

```
osbams/
├── README.md
├── LICENSE
├── CHANGELOG.md
├── pyproject.toml
├── .gitignore
├── desktop/
│   ├── osbams/
│   │   ├── app/
│   │   ├── domain/
│   │   ├── services/
│   │   ├── repositories/
│   │   ├── protocols/
│   │   ├── reports/
│   │   └── config/
│   ├── tests/
│   └── main.py
├── firmware/
│   ├── Core/
│   ├── App/
│   ├── Drivers/
│   ├── Middleware/
│   ├── Platform/
│   ├── Tests/
│   ├── CMakeLists.txt
│   └── OSBAMS.ioc
├── hardware/
│   ├── schematic/
│   ├── pcb/
│   ├── enclosure/
│   ├── bom/
│   ├── wiring/
│   └── calculations/
├── protocol/
│   └── protocol_spec.md
```



Only one file of each functional implementation should exist.

3. Replace INA260 completely in the production path

The final product should not contain an active INA260 measurement path for a 42 V pack.

Replace:

`ina260.c`
`ina260.h`

with:

`ina228.c`
`ina228.h`

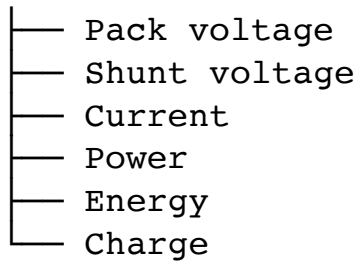
Update:

- Sensor Manager
- Device detection
- Calibration

- Fault reporting
- Documentation
- Schematics
- Protocol metadata
- Test procedures
- BOM
- Risk register
- Traceability matrix

The final design should use:

INA228



Keep the STM32 ADC divider only as a **redundant independent voltage channel**, not as the primary sensor.

The existing documentation still identifies the INA260's 36 V limit as a major blocker, so the production documentation must be revised once INA228 is implemented and validated.

4. Design a real power and safety system

For real-world use, software shutdown alone is insufficient.

Main power path

Battery positive

- keyed connector
- reverse-polarity protection or preconnection check
- DC-rated branch fuse
- manual service disconnect
- normally-open DC contactor
- precision shunt / INA228
- electronic load
- battery negative

Independent control path

Isolated 12 V control supply

- control fuse
- key/ARM switch
- E-stop NC contact
- hardware safety interlock
- contactor coil
- MOSFET driver

The STM32 may command the MOSFET, but pressing the E-stop must remove contactor-coil power even if:

- Firmware crashes
- USB disconnects
- The desktop freezes
- FreeRTOS deadlocks
- The GPIO remains asserted

Feedback inputs

Add:

- Contactor auxiliary-contact feedback
- E-stop status
- Door/enclosure interlock
- Load-enable feedback
- Control-supply-good signal
- Pack-present input
- Fuse-status detection if practical

Add fault codes:

CONTACTOR_WELDED
CONTACTOR_FAILED_TO_CLOSE
ESTOP_ACTIVE
INTERLOCK_OPEN
LOAD_FEEDBACK_MISMATCH
CONTROL_POWER_LOW
PACK_REVERSED
PACK_OVERVOLTAGE

5. Add precharge and inrush control where necessary

Depending on the electronic load's input capacitance, connecting a pack directly may create an inrush spark.

Add a precharge path:

Battery

- precharge resistor
- precharge relay or MOSFET
- load input capacitance

Sequence:

1. Manual disconnect on.
2. Verify polarity.
3. Close precharge path.
4. Wait for bus voltage to settle.
5. Confirm voltage difference is small.
6. Close main contactor.
7. Open precharge path.
8. Enable the electronic load.

Whether this is required depends on the selected load, but a production system should explicitly evaluate it.

6. Use a properly rated electronic load

For a market-like unit, I would not make manual load adjustment the final design.

Use a programmable load with:

- At least 60 V input
- At least 10 A
- Preferably 300–600 W
- CC, CP and CR modes
- Remote enable
- SCPI over USB, Ethernet, RS-232 or LAN
- Front-panel lockout
- Protection status query
- Measurement readback

Create instrument drivers:

```
desktop/osbams/instruments/  
├── base_load.py  
├── manual_load.py  
├── scpi_load.py  
├── owon_load.py  
└── simulator.py
```

The production sequence should be:

```
Desktop sends test configuration  
→ STM32 verifies hardware readiness  
→ load receives setpoint  
→ STM32 closes contactor  
→ load output is enabled  
→ firmware independently monitors safety
```

The STM32 must still be capable of opening the contactor without the desktop or load driver.

7. Complete the firmware instead of retaining future stubs

Your current documentation identifies several unimplemented peripherals.

For the final version, each feature must be either:

- Fully implemented and tested, or
- Removed from the product requirements and source tree

Do not ship visible placeholders.

Minimum production firmware

Implement:

Deterministic acquisition

TIM6

- ISR/task notification
- Sensor acquisition
- Timestamped sample
- Measurement queue

Do not use a general-purpose polling delay for critical sampling.

FreeRTOS

Use a small task set:

SafetyTask	highest priority
AcquisitionTask	
TestControllerTask	
ProtocolTask	
StorageTask	
UiTask	lowest priority

Keep it restrained. Too many tasks add failure modes.

Watchdog

Use the independent watchdog.

Each critical task reports health. Refresh only when all required tasks have executed within the supervision interval.

On reset:

- Keep contactor off
- Record reset cause
- Mark interrupted test incomplete
- Require explicit operator acknowledgement

Nonblocking communication

Replace blocking UART with:

- DMA TX queue
- DMA RX or USB CDC receive buffering
- Bounded frame sizes
- Parser timeouts
- Overflow handling

Local storage

Add microSD or durable internal flash logging so a host disconnect does not lose a long test.

Log:

TEST_HEADER
SAMPLE
STATE_CHANGE
WARNING
FAULT
TEST_SUMMARY
RESET_CAUSE

Calibration storage

Store calibration values in internal flash with:

- Structure version
- Hardware serial
- Date
- Slope
- Offset
- CRC
- Validity flag

Boot self-test

At startup:

1. Verify configuration CRC.
2. Verify load disabled.
3. Verify contactor feedback open.
4. Verify E-stop released.
5. Verify mandatory sensors.
6. Verify storage.
7. Verify supply voltage.
8. Verify watchdog/reset cause.
9. Enter IDLE only when safe.

8. Improve temperature sensing

A single TC74 with 1 °C resolution is acceptable for a prototype, but weak for a market-like battery test fixture.

Use at least:

T1 — pack surface
T2 — connector/contact area
T3 — enclosure ambient
T4 — load or shunt region

Potential options:

- Digital temperature sensors
- Thermistors with protected ADC channels
- Thermocouple interface for validation
- RTDs for higher accuracy

Also implement:

- Absolute overtemperature
- Temperature-rise rate
- Sensor-disagreement detection
- Missing-sensor fault
- Connector-overtemperature cutoff

For example:

Pack temperature $\geq$ limit
or
temperature rise $\geq$ configured °C/min
or
connector temperature $\geq$ connector limit
→ terminate discharge

9. Add redundant voltage safety

The INA228 should provide normal measurements, but an independent voltage safety channel improves fault tolerance.

Use:

Protected resistor divider
→ RC filter
→ STM32 ADC

The ADC channel does not need metrology-grade accuracy. Its purpose is to detect:

- Gross overvoltage
- Gross undervoltage
- INA228 disagreement
- Sensor bus failure

Example:

```
abs(INA228 voltage - ADC voltage) > permitted difference
→ sensor disagreement fault
→ open contactor
```

10. Redesign the firmware state machine around real hardware

Use a final state machine such as:

```
POWER_OFF
BOOT
SELF_TEST
SAFE_IDLE
PACK_DETECTED
POLARITY_CHECK
PRECHARGE
PACK_READY
TEST_CONFIGURED
RESTING
DISCHARGING
PAUSED
CUTOFF
RECOVERY
COMPLETE
FAULT_LATCHED
SERVICE
```

Important rules:

- No direct jump from IDLE to DISCHARGING.
- Contactor closure requires successful prechecks.
- Load enable requires contactor feedback.

- Any critical fault immediately disables the load and opens the contactor.
- A fault cannot be cleared while its cause remains active.
- Restart after a fault requires explicit acknowledgement.

11. Make the protocol production-grade

Keep one protocol implementation in C and one matching decoder/encoder in Python.

I would use a versioned message format with explicit types:

```
OSBAMS, 1, DATA, ...
OSBAMS, 1, STATUS, ...
OSBAMS, 1, EVENT, ...
OSBAMS, 1, FAULT, ...
OSBAMS, 1, COMMAND, ...
OSBAMS, 1, ACK, ...
OSBAMS, 1, NACK, ...
OSBAMS, 1, SUMMARY, ...
```

Add:

- Protocol version
- Message type
- Sequence
- Timestamp
- Payload length or deterministic field count
- CRC
- Command transaction ID
- ACK/NACK reason codes
- Maximum frame length
- Parser timeout
- Sequence rollover definition
- Timestamp rollover definition

Commands should include:

```
GET_STATUS
GET_VERSION
GET_CALIBRATION
SET_TEST_CONFIG
ARM
START
PAUSE
```

RESUME
STOP
ACK_FAULT
SET_RTC
DOWNLOAD_LOG

Never let a command directly bypass safety-state conditions.

12. Use one version and configuration authority

Create a generated build-information system.

Desktop

APP_VERSION
DATABASE_SCHEMA_VERSION
RULE_ENGINE_VERSION
PROTOCOL_VERSION

Firmware

FIRMWARE_VERSION
PROTOCOL_VERSION
HARDWARE_REVISION
BUILD_COMMIT
BUILD_TIMESTAMP

Generate build metadata during compilation rather than manually editing several files.

Every report should include:

App version
Firmware version
Protocol version
Database schema

Rule engine version
Model version
Hardware revision
Git commit
Calibration record

13. Redesign the database into one authoritative schema

Do not keep creation logic distributed between:

- `database.py`
- `migrations.py`
- `ensure_features_table()`
- service modules

Create one baseline schema and ordered migrations.

Recommended tables:

`assets`
`asset_inspections`
`tests`
`test_configurations`
`readings`
`test_events`
`test_summaries`
`features`
`assessments`
`suitability_results`
`calibrations`
`instruments`
`model_versions`
`audit_log`
`schema_info`

Important improvements:

- Use SQLite backup API.
- Back up before migration.

- Batch reading inserts.
- Add indexes.
- Use transactions.
- Use foreign-key cascades deliberately.
- Whitelist update fields.
- Use `ON CONFLICT DO UPDATE`, not `INSERT OR REPLACE`.
- Store timestamps consistently in UTC.
- Store test configuration snapshots.
- Mark data origin as real, simulated or imported.
- Never mix simulated data into production model training silently.

14. Refactor desktop code into layers

The final GUI should not directly manage SQL, serial parsing, scoring and files from widget callbacks.

Use:

GUI

- Application services
- Domain models
- Repositories/instrument drivers

Suggested classes:

```
BatteryService
TestService
SafetyService
InstrumentService
AssessmentService
ReportService
CalibrationService
DatabaseRepository
FirmwareClient
```

Use typed dataclasses for:

```
BatteryAsset
TelemetrySample
TestConfiguration
```

TestSummary
SafetyStatus
AssessmentResult
CalibrationRecord
InstrumentIdentity

This makes the package easier to test and maintain.

15. Normalize all statuses and enums

Use one exact canonical set.

Safety

PASS
HOLD
QUARANTINE
FAIL

Test result

COMPLETE
OPERATOR_STOP
SAFETY_STOP
SENSOR_FAILURE
HOST_DISCONNECTED
POWER_FAILURE
INCOMPLETE

Data quality

VALID
PARTIAL
INVALID
SIMULATED

Sensor health

ONLINE
DEGRADED
OFFLINE
NOT_FITTED

Do not allow arbitrary status strings in the database or UI.

16. Improve input and start-test validation

Before allowing ARM or START, verify:

- Known battery identity
- Chemistry selected
- Series count reasonable
- Pack voltage within profile
- Polarity correct
- Fuse installed
- E-stop released
- Door closed
- Contactor feedback open
- INA228 online
- Required temperature sensors online
- Load connected and rated
- Load setpoint within limits
- Cutoff compatible with chemistry
- Current and power within wiring, load and pack limits
- Calibration current
- Storage available
- No unresolved fault
- Pack not marked QUARANTINE or FAIL

Use hard errors for unsafe conditions, not warnings.

17. Improve the assessment design

The deterministic rule engine should remain the primary assessment because it is explainable and defensible.

For the ML portion:

- Train only on real batteries unless explicitly benchmarking.
- Use grouped validation by battery ID.
- Store label origin.
- Store dataset hash.
- Store model parameters and feature list.
- Display independent battery count and total test count.
- Avoid claiming independent predictive accuracy when labels come from the rule engine.

Your limitations document already states this problem clearly.

For a market-like product, ML should remain advisory until externally validated ground truth is available.

18. Build actual hardware documentation

Create:

```
hardware/
├── system_schematic.pdf
├── controller_schematic.pdf
├── power_path_schematic.pdf
├── wiring_harness.pdf
├── connector_pinout.pdf
├── bom.xlsx
├── fuse_calculation.md
├── wire_derating.md
├── shunt_calculation.md
├── thermal_analysis.md
├── creepage_clearance_review.md
└── enclosure_drawing.pdf
```

The BOM must specify exact:

- Manufacturer
- Part number
- Voltage rating
- Current rating
- Temperature rating
- Flammability rating
- Quantity
- Supplier
- Approved substitute

19. Build a custom PCB

For a market-quality version, do not use loose jumper wires or a permanent breadboard/perfboard assembly.

A controller PCB should include:

- STM32 or production MCU module
- INA228 interface
- Shunt interface
- ADC divider
- Temperature inputs
- Isolated or protected communication
- MOSFET contactor driver
- Feedback inputs
- E-stop/interlock inputs
- Power regulation
- Fusing
- Transient suppression
- Reverse-polarity protection
- ESD protection
- Labeled connectors
- Test points
- Programming header

Separate:

high-energy wiring
low-voltage logic
measurement analog
communication

Use proper creepage, clearance and grounding strategy.

20. Add electrical protection

Include engineering review for:

- DC fuse interrupt rating
- TVS suppression
- Reverse-polarity protection
- Input transient protection
- Coil flyback or bidirectional suppression

- Shunt Kelvin connections
- Load-side transient behavior
- USB ESD protection
- I²C protection if sensors are remote
- Ground-loop control
- Connector touch safety
- Cable strain relief
- Flame-rated enclosure
- Covered terminals

Do not depend on an ordinary fuse whose DC interrupt rating is undocumented.

21. Add instrument identification

At test start, record:

- Electronic-load manufacturer/model/serial
- Reference DMM
- Controller serial
- Hardware revision
- Calibration due dates
- Firmware version
- Load firmware if available

This belongs in the database and reports.

22. Add full calibration support

Calibrate:

- INA228 bus voltage
- Current/shunt measurement
- Redundant ADC voltage
- Each temperature channel
- Electronic load readback if used

Use multiple points over the operating range.

Store:

```
slope
offset
residuals
reference instrument
reference serial
```

calibration date
calibration due date
operator
environment

Do not advertise measurement accuracy until data supports it, consistent with your system specification.

23. Add formal verification and validation

A product-quality suite needs evidence, not only code and architecture.

Unit tests

- Measurement math
- Safety thresholds
- State transitions
- Protocol
- Validators
- Database
- Scoring
- Suitability

Hardware-in-the-loop tests

- Simulated voltage/current sensor inputs
- Temperature fault injection
- I2C disconnect
- Contactor feedback mismatch
- E-stop activation
- Load communication failure
- Host disconnect
- Storage failure
- Watchdog reset
- Power interruption

Endurance tests

- 8-hour continuous test
- Repeated reconnects
- Thousands of protocol frames
- Database growth test
- Multiple battery sessions
- Power-cycle recovery

Accuracy tests

- Voltage calibration
- Current calibration
- Temperature calibration
- Ah comparison
- Wh comparison
- Timing accuracy
- Repeatability

Your existing specification already provides a useful acceptance-test foundation, but the final product must record actual results rather than leaving every hardware-validation entry as “No.”

24. Add production diagnostics

Create a service mode that can:

- Scan sensors
- Read raw registers
- Exercise contactor
- Verify feedback
- Read E-stop
- Check storage
- Test buzzer/LED/display
- Display ADC values
- Verify load communication
- Export diagnostic bundle
- Show reset cause
- Show calibration state

Service mode must prohibit unsafe load operation without appropriate controls.

25. Add local user interface

A real standalone instrument should not require the PC for every safety-relevant action.

Minimum front panel:

- Emergency stop
- ARM switch
- Start
- Stop
- Acknowledge
- Status LEDs
- Small display

Display:

Pack voltage
Current
Temperature
State
Elapsed time
Ah
Wh
Fault

The desktop remains the advanced interface, but the controller should continue safely without it.

26. Improve enclosure design

Use an enclosure large enough to provide:

- Terminal covers
- Separation of power and logic
- Airflow
- Cable bend radius
- Service access
- Contact protection
- Clear labels
- Strain relief
- Grounding where required

Label:

BATTERY INPUT
LOAD OUTPUT
MAX PACK VOLTAGE
MAX CURRENT
E-STOP
ARM
USB
CONTROL POWER
FUSE RATING

Add visible hazard and polarity markings.

27. Clean the final documentation

The production package should not describe three generations.

The final documents should describe only the final product.

Keep historical development in Git history and perhaps the capstone report, not in operational documentation.

Recommended documentation:

README

System Specification

Hardware Design

Firmware Architecture

Desktop Architecture

Protocol Specification

User Manual

Service Manual

Calibration Procedure

Validation Report

Known Limitations

Release Notes

Remove obsolete claims about:

- INA260 being the active sensor
- Planned ADC as the primary solution
- Manual load if remote control is implemented
- Unimplemented peripherals that are no longer in scope
- Old firmware versions
- Old database schemas

28. Add software quality tooling

Use:

Python

```
|— ruff
|— black
```

- mypy or pyright
- pytest
- coverage

C

- clang-format
- cppcheck
- clang-tidy
- compiler warnings as errors
- host unit tests

Add continuous integration that:

1. Lints Python.
2. Runs Python tests.
3. Builds firmware host tests.
4. Runs C tests.
5. Validates protocol vectors.
6. Checks documentation links.
7. Builds a release package.

29. Produce one reproducible release

A clean checkout should allow:

```
make test
make desktop
make firmware
make docs
make package
```

The release artifact should contain:

```
Desktop installer
Firmware binary
Firmware checksum
Protocol specification
User manual
Hardware revision
Database migration tool
```

Release notes

30. Understand “market-ready”

A fully engineered prototype and a legally marketable commercial product are not the same.

Before commercial sale, you may need professional evaluation for applicable areas such as:

- Electrical safety
- EMC emissions and immunity
- FCC requirements
- Product liability
- Battery-test safety procedures
- Environmental compliance
- UL/IEC standards depending on product classification
- Enclosure flammability
- Labelling and instructions
- Calibration traceability

Do not claim certification until it has actually been completed.

Exact implementation order

Phase 1 — Clean architecture

1. Tag and archive the old versions in Git.
2. Create one clean production branch.
3. Delete duplicate and scaffold files.
4. Normalize folders, versions, statuses and configuration.
5. Establish one database schema.
6. Establish one protocol.

Phase 2 — Final hardware design

7. Select final electronic load.
8. Finalize INA228 and shunt.
9. Finalize contactor, E-stop and control supply.
10. Add precharge if needed.
11. Add redundant ADC voltage sensing.
12. Add multiple temperature sensors.
13. Complete schematic, calculations and BOM.
14. Design PCB and enclosure.

Phase 3 — Final firmware

15. Implement INA228 driver.
16. Complete I²C driver.
17. Implement deterministic timer acquisition.
18. Integrate FreeRTOS.
19. Implement watchdog.
20. Implement contactor/interlock driver.
21. Implement load control.
22. Implement local storage.
23. Implement USB communication.
24. Complete protocol commands.
25. Implement calibration storage.
26. Complete boot self-test and diagnostics.

Phase 4 — Desktop cleanup

27. Refactor GUI logic into services.
28. Finish typed data models.
29. Optimize database transactions.
30. Add instrument and calibration management.
31. Add test-quality classification.
32. Complete automated test orchestration.
33. Update reports and reproducibility metadata.

Phase 5 — Verification

34. Run static analysis.
35. Run unit and integration tests.
36. Calibrate all channels.
37. Perform fault injection.
38. Perform endurance tests.
39. Complete every applicable acceptance test.
40. Publish the validation report.

Phase 6 — Release

41. Remove all obsolete documentation.
42. Freeze hardware Rev A.
43. Freeze protocol v1.
44. Freeze database schema.
45. Produce firmware and desktop installers.
46. Tag the release.
47. Keep all subsequent changes in a new development branch.

The most important principle is:

Do not carry “experimental firmware” forward as a separate edition. Integrate only the features that pass verification into one final production firmware. Remove or defer everything else.

That gives you the clean, optimized, single-version package you are describing.